Chapter 23 | Exercise solutions with Python

Vision–Language and Multimodal Models

This notebook solves Exercises 5–8. It checks how normalization and scaling affect image–text scores, how a transpose exchanges the two contrastive losses, and how relevance and attention behave when the two sides have different lengths.

Each section uses the chapter's local NumPy implementation with supplied vectors. The examples have small enough arrays to check each result by hand. The written solutions cover Exercises 1–4 and 9–11.

To run the notebook, open it inside a copy of `transformers-book` and select a Python kernel with the packages in `requirements.txt`. Run the cells in order. The exercises run on CPU and include their saved outputs.

```
from pathlib import Path
import sys

import numpy as np

working_directory = Path.cwd()
for repo_root in (working_directory, *working_directory.parents):
    model_path = (
        repo_root / "examples" / "ch23_multimodal"
    )
    if model_path.is_dir():
        sys.path.insert(0, str(repo_root))
        break
else:
    raise FileNotFoundError(
        "Open this notebook inside transformers-book."
    )
```

5. Check row normalization

Start with the image and text vectors from Exercise 1. Stretch the first image vector from $(3, 0)$ to $(15, 0)$. Both point in the same direction. Dividing each by its length gives $(1, 0)$ in either case.

For a nonzero row v and a positive multiplier c, the Euclidean length scales by c. This makes the unit row invariant, meaning it stays the same after the stretch.

$$\frac{cv}{\| cv \|_2} = \frac{cv}{c \| v \|_2} = \frac{v}{\| v \|_2}.$$

The normalized image rows are $(1, 0)$ and $(0, 1)$. The normalized text rows are $(1/\sqrt{2}\ , 1/\sqrt{2}\ )$ and $(0, 1)$. Taking every image–text dot product and multiplying by 2 gives the logits, or scores before softmax. At scale 4, the same dot products get twice the score.

$$L_2 = \begin{bmatrix} \sqrt{2} & 0 \\ \sqrt{2} & 2 \end{bmatrix}, \qquad L_4 = \begin{bmatrix} 2\sqrt{2} & 0 \\ 2\sqrt{2} & 4 \end{bmatrix}.$$

The first image ranks text 1 first, and the second image ranks text 2 first. Multiplying every score by the same positive number preserves inequalities and ties. This applies to rankings along rows and columns.

```
from examples.ch23_multimodal.model import contrastive_logits

images = np.array([[3.0, 0.0], [0.0, 2.0]])
texts = np.array([[1.0, 1.0], [0.0, 2.0]])
stretched_images = images.copy()
stretched_images[0] *= 5

original = contrastive_logits(images, texts, scale=2)
stretched = contrastive_logits(stretched_images, texts, scale=2)
rescaled = contrastive_logits(stretched_images, texts, scale=4)

root_two = np.sqrt(2)
expected_at_two = np.array([[root_two, 0], [root_two, 2]])
expected_text_order = [[1, 2], [2, 1]]
cases = [
    ("Original images, scale 2", original, expected_at_two),
    ("Stretched first image, scale 2", stretched, expected_at_two),
    ("Stretched first image, scale 4", rescaled, 2 * expected_at_two),
]
for label, scores, expected in cases:
    np.testing.assert_allclose(scores, expected)
    text_order = np.argsort(-scores, axis=1, kind="stable") + 1
    np.testing.assert_array_equal(text_order, expected_text_order)
    print(label)
    print(np.array2string(scores, precision=6, floatmode="fixed"))
    print(f"Text order by image (1-based) {text_order.tolist()}")
    paired_probability = 1 / (1 + np.exp(scores[0, 1] - scores[0, 0]))
    print(f"P(text 1 | image 1) = {paired_probability:.6f}")
```

OUTPUT

```
Original images, scale 2
[[1.414214 0.000000]
 [1.414214 2.000000]]
Text order by image (1-based) [[1, 2], [2, 1]]
P(text 1 | image 1) = 0.804430
Stretched first image, scale 2
[[1.414214 0.000000]
 [1.414214 2.000000]]
Text order by image (1-based) [[1, 2], [2, 1]]
P(text 1 | image 1) = 0.804430
Stretched first image, scale 4
[[2.828427 0.000000]
 [2.828427 4.000000]]
Text order by image (1-based) [[1, 2], [2, 1]]
P(text 1 | image 1) = 0.944193
```

Softmax turns a row of scores into probabilities by exponentiating and dividing by the sum of the exponentials. The first row has two candidates, so the probability of text 1 simplifies to $1/(1 + e^{L_{12} - L_{11}})$, using one-based indices. Doubling the scale doubles the gap between the two scores. This raises the probability from about 0.804 to 0.944 while text 1 stays in first place.

6. Transpose a contrastive batch

Use the scores from Exercise 2. Each row starts as an image query against two text candidates. Each column is a text query against two image candidates. The two supplied pairs occupy the diagonal.

$$L = \begin{bmatrix} 1 & 0 \\ 2 & 3 \end{bmatrix}, \qquad L^{\mathsf{T}} = \begin{bmatrix} 1 & 2 \\ 0 & 3 \end{bmatrix}.$$

For each query, the loss is the negative natural log of the softmax probability of its paired candidate. With two candidates, let d be the paired score minus the other score. The paired probability is $1/(1 + e^{-d})$, so the loss is $\ln(1 + e^{-d})$. Average the two query losses for each direction.

- Across rows, the gaps are $1 - 0 = 1$ and $3 - 2 = 1$. The image-to-text loss is $\ln(1 + e^{-1}) \approx 0.313262$.
- Down columns, the gaps are $1 - 2 = -1$ and $3 - 0 = 3$. The text-to-image loss is $[\ln(1 + e) + \ln(1 + e^{-3})]/2 \approx 0.680925$.

The mean of the two directional losses is about 0.497093. The code calls the chapter function on both matrices and compares its return values with this arithmetic.

```
from math import exp, log1p

from examples.ch23_multimodal.model import symmetric_contrastive_loss

logits = np.array([[1.0, 0.0], [2.0, 3.0]])
row_loss, column_loss, mean_loss = symmetric_contrastive_loss(logits)
transpose_row, transpose_column, transpose_mean = (
    symmetric_contrastive_loss(logits.T)
)

expected_row = log1p(exp(-1))
expected_column = (log1p(exp(1)) + log1p(exp(-3))) / 2
expected_mean = (expected_row + expected_column) / 2
np.testing.assert_allclose(
    (row_loss, column_loss, mean_loss),
    (expected_row, expected_column, expected_mean),
)
np.testing.assert_allclose(
    (transpose_row, transpose_column, transpose_mean),
    (expected_column, expected_row, expected_mean),
)
```

```
loss_cases = [
    ("L", row_loss, column_loss, mean_loss),
    ("L.T", transpose_row, transpose_column, transpose_mean),
]
print(f"{'Input':<6} {'First (rows)':>13} "
      f"{'Second (cols)':>13} {'Mean':>10}")
for label, first, second, mean in loss_cases:
    print(f"{label:<6} {first:>13.6f} {second:>13.6f} {mean:>10.6f}")
```

```
OUTPUT
Input   First (rows) Second (cols)       Mean
L           0.313262      0.680925   0.497093
L.T         0.680925      0.313262   0.497093
```

The transpose turns each original column into a row and each original row into a column. It keeps each paired score at the same diagonal position. The first returned loss therefore compares the original images for each text after transposition. The second loss compares the original texts for each image. Exchanging the two losses leaves their arithmetic mean unchanged.

7. Build a rectangular retrieval task

There are two images, I1 and I2, and three captions, C1 through C3. The score and relevance matrices both have shape 2×3. A score determines where a candidate ranks. The Boolean relevance entry says whether that candidate counts as a correct result. Here C1 and C2 are valid for I1, and C3 is valid for I2.

To retrieve captions, rank each image's row from highest to lowest score. To retrieve images, rank each caption's column. The same relevance entries apply in both directions. For each query, recall at k divides the number of relevant items in the first k results by the number of relevant items for that query.

Query	Candidates in score order	Relevant candidates	Recall at 1	Recall at 2
I1	C1, C2, C3	C1, C2	1/2	2/2
I2	C3, C2, C1	C3	1/1	1/1
C1	I1, I2	I1	1/1	1/1
C2	I2, I1	I1	0/1	1/1
C3	I2, I1	I2	1/1	1/1

For I1 at rank one, C1 accounts for only half of the two valid captions. For C2, the scores are 1 for I1 and 2 for I2, so the only relevant image appears at rank two. Each mean gives equal weight to every query with defined recall. At $k = 1$ the means are $(1/2 + 1)/2 = 3/4$ for image-to-text and $(1 + 0 + 1)/3 = 2/3$ for text-to-image. At $k = 2$ every query has retrieved all of its relevant items, so both means are 1.

```
from examples.ch23_multimodal.model import bidirectional_recall_at_k

scores = np.array([[2, 1, 0], [0, 2, 3]])
relevance = np.array([[1, 1, 0], [0, 0, 1]], dtype=bool)

# Expected per-query recall and mean from the hand calculation.
expected_recall = {
    1: {
        "image_to_text": ([1 / 2, 1], 3 / 4),
        "text_to_image": ([1, 0, 1], 2 / 3),
    },
    2: {
        "image_to_text": ([1, 1], 1),
        "text_to_image": ([1, 1, 1], 1),
    },
}

print(f"{'k':>1}  {'Direction':<13} {'per_query':<17} {'mean':>8}")
for k, expected_directions in expected_recall.items():
    recall = bidirectional_recall_at_k(scores, relevance, k)
    for direction, expected in expected_directions.items():
        result = recall[direction]
        per_query, mean = result["per_query"], result["mean"]
        expected_per_query, expected_mean = expected
        np.testing.assert_allclose(per_query, expected_per_query)
        np.testing.assert_allclose(mean, expected_mean)
        assert result["defined_queries"] == len(expected_per_query)

        label = direction.replace("_", "-")
        print(f"{k:>1}  {label:<13} {str(per_query):<17} "
              f"{mean:>8.6f}")
```

```
OUTPUT
k  Direction     per_query            mean
1  image-to-text [0.5, 1.0]        0.750000
1  text-to-image [1.0, 0.0, 1.0]   0.666667
2  image-to-text [1.0, 1.0]        1.000000
2  text-to-image [1.0, 1.0, 1.0]   1.000000
```

C2 contributes a defined zero to the text-to-image mean at $k = 1$ because it has a relevant image outside the top result. Every caption has a relevant image in this exercise. If we removed the C2 relevance mark for I1, its column would contain all `False`. The denominator would be zero, so the function would return `None` for C2 and average the two remaining caption queries.

The given scores have no tie within any query's ranking. When two candidates for the same query have equal scores, the function preserves their order in the input. For example, if both images scored 1 for C2 and the original relevance labels were used, I1 would come first and C2 would have recall 1 at rank one.

8. Change what a query reads (optional)

Think of the two key and value rows as two image regions. The query scores each region by comparing with its key. The attention weights then determine how much of each region's value enters the context row. We keep the same keys, values, and projection for both runs.

The query $(0, \sqrt{2})$ has a dot product of 0 with the first key and $\sqrt{2}\ \ln 2$ with the second. Dividing by $\sqrt{d_k} = \sqrt{2}$, where d_k is the key width, gives $(0,\ \ln 2)$. Softmax exponentiates the two scores and divides each exponential by their sum. The exponentials $(1, 2)$ become weights $(1/3, 2/3)$. The second value gets twice the weight of the first. The weighted sum is

$$\tfrac{1}{3}(3, 0) + \tfrac{2}{3}(0, 3) = (1, 2).$$

With the negated query $(0,\ -\sqrt{2})$, the scaled scores are $(0,\ -\ \ln 2)$. The exponentials $(1, 1/2)$ now give weights $(2/3, 1/3)$, so the first value gets twice the weight of the second. The new context is

$$\tfrac{2}{3}(3, 0) + \tfrac{1}{3}(0, 3) = (2, 1).$$

The projection sends a context row (a, b) to $(a, b, a - b)$. Multiply a by the first projection row and b by the second, then add them. The two projected rows are therefore $(1, 2,\ -1)$ and $(2, 1, 1)$. Each run starts with one query row of width two. Its two weights produce one context row of width two. Multiplying by the 2×3 projection gives one row of width three.

```
from examples.ch23_multimodal.model import image_feature_bridge

query = np.array([[0, np.sqrt(2)]])
keys = np.array([[0, 0], [0, np.log(2)]])
values = np.array([[3, 0], [0, 3]])
projection = np.array([[1, 0, 1], [0, 1, -1]])

# The hand calculation gives one row for each expected array.
bridge_cases = [
    ("Original", query, np.log(2), [[1 / 3, 2 / 3]], [[1, 2]],
     [[1, 2, -1]]),
    ("Negated", -query, -np.log(2), [[2 / 3, 1 / 3]], [[2, 1]],
     [[2, 1, 1]]),
]

assert query.shape == (1, 2)
assert keys.shape == values.shape == (2, 2)
assert projection.shape == (2, 3)
for label, active_query, log_score, want_weights, want_ctx, want in (
    bridge_cases
):
    context, weights, projected = image_feature_bridge(
        active_query, keys, values, projection
    )
    scaled_scores = active_query @ keys.T / np.sqrt(keys.shape[1])
    np.testing.assert_allclose(scaled_scores, [[0, log_score]])
    np.testing.assert_allclose(weights, want_weights)
    np.testing.assert_allclose(context, want_ctx)
    np.testing.assert_allclose(projected, want)

    print(f"{label} query")
    print(f"  Scaled scores  {np.round(scaled_scores, 6)}")
    print(f"  Weights        {np.round(weights, 6)}")
    print(f"  Context        {context}")
    print(f"  Projected row  {projected}")

print(f"Shapes  weights {weights.shape}, context {context.shape}, "
      f"projection output {projected.shape}")
```

OUTPUT

```
Original query
  Scaled scores  [[0.       0.693147]]
  Weights        [[0.333333 0.666667]]
  Context        [[1. 2.]]
  Projected row  [[ 1.  2. -1.]]
Negated query
  Scaled scores  [[-0.       -0.693147]]
  Weights        [[0.666667 0.333333]]
  Context        [[2. 1.]]
  Projected row  [[2. 1. 1.]]
Shapes  weights (1, 2), context (1, 2), projection output (1, 3)
```